Pontifícia Universidade Católica de Goiás
Pontifical Catholic University of Goias
Av. Universitária, 1069, Setor Universitário
Caixa Postal 86 - CEP 74.605-010
Goiânia – Goiás – Brasil

Relatório Técnico-Acadêmico do Projeto Integrador III-B

Professor: Thalles Santos

Alunos: Luciana Lopes de Freitas, Kassia Soares Darcie Leão e Gustavo Oliveira Arantes

---

Contextualização Extensionista

Conforme a proposta do Projeto Integrador III-B, o objetivo deste trabalho foi desenvolver uma solução web aplicada ao contexto jurídico do escritório Moura Advocacia, atendendo uma necessidade real de organização, acompanhamento e apresentação institucional.

O projeto foi direcionado à rotina de um escritório de advocacia, com foco na centralização das informações, no acompanhamento de processos e no apoio ao relacionamento com clientes por meio de uma aplicação web funcional e de fácil uso.

No escopo deste trabalho, a solução foi organizada para responder a demandas concretas de gestão de casos e de comunicação institucional. Essa aplicação prática se consolida ao:

- Atender uma necessidade real de centralização de informações e rastreabilidade das atividades;
- Reduzir a dispersão de dados em planilhas, anotações e controles informais;
- Favorecer o controle de prazos, etapas e informações cadastrais;
- Aproximar os conhecimentos acadêmicos de engenharia de software de uma necessidade aplicada do escritório.

  Sob a ótica de utilidade prática, a melhoria da organização interna contribui para previsibilidade, agilidade e melhor resposta às demandas do serviço jurídico, reforçando o caráter extensionista do projeto.

Descrição da Solução Proposta

A solução desenvolvida é composta por três frentes principais:

- **API backend** em Java com Spring Boot para autenticação, usuários e clientes;
- **Interface Kanban** em React + Vite para organização visual dos processos jurídicos;

- **Landing page institucional** em HTML e CSS para apresentação profissional do escritório.

  Em termos práticos, o sistema busca resolver o problema de desorganização e baixa visibilidade sobre o andamento das atividades, oferecendo:

- Autenticação e controle de acesso;
- Cadastro e gestão de clientes;
- Acompanhamento visual de processos em quadro Kanban;
- Operações de criação, atualização, movimentação e exclusão de cartões;
- **Persistência local dos cartões do Kanban via localStorage**, permitindo uso offline da interface.

Metodologia de Desenvolvimento Adotada

A metodologia adotada combinou desenvolvimento incremental, organização modular e validação contínua das funcionalidades, com foco em entregas funcionais e no alinhamento entre backend e frontend.

No desenvolvimento do repositório, a metodologia foi materializada por meio das seguintes práticas:

- Separação entre backend, frontend institucional e frontend Kanban;
- Organização do backend em módulos `bootstrap`, `user` e `customer`, com inspiração em arquitetura hexagonal;
- Construção de protótipos funcionais no frontend para validação da experiência de uso;
- Versionamento do código em Git e estrutura preparada para execução local e conteinerizada.

Tecnologias e Padrões Empregados

Backend

- **Java 21**
- **Spring Boot 3.2.4**
- **Spring Security** com JWT para autenticação stateless
- **Spring Data JPA** para persistência

- **PostgreSQL** como banco de dados
- **Maven** multi-módulo para organização de dependências

Frontend Kanban

- **React 18** com TypeScript
- **Vite** como bundler
- **Tailwind CSS** para estilização
- **React Router** para navegação
- **React Hook Form** para formulários
- **localStorage** para persistência local dos dados
- Componentes reutilizáveis e interação por **drag-and-drop**

Frontend Institucional

- **HTML5** semântico
- **CSS3** puro com variáveis CSS
- **JavaScript** nativo

Infraestrutura

- **Dockerfile** multi-stage para otimização de imagem
- **Docker Compose** para orquestração local da API e banco de dados

  Essa combinação tecnológica evidencia uma implementação consistente de sistema web, com separação clara de responsabilidades e base técnica adequada para evolução contínua.

Gestão do Projeto

A gestão do projeto foi estruturada com mapeamento de tarefas, definição de responsabilidades e acompanhamento contínuo do progresso por meio de quadro visual.

O acompanhamento operacional foi realizado no Trello, disponível em: https://trello.com/b/P00CHXuR/projetoiiib.

No repositório, identificam-se elementos concretos de organização do trabalho:

- Estrutura modular por domínio técnico:
  - `api/bootstrap` - inicialização da aplicação

    - `api/user` - autenticação e usuários
    - `api/customer` - gestão de clientes
    - `view/landing` - landing page institucional
    - `view/kanban` - interface Kanban jurídica
- Separação entre camadas funcionais e de apresentação, facilitando manutenção e evolução;
- Documentação técnica nos READMEs da raiz e da API, orientando execução e entendimento do sistema;
- Processo de integração local com containers, favorecendo reprodutibilidade do ambiente;
- Quadro visual de acompanhamento do fluxo de trabalho, apoiando planejamento e priorização das entregas.

Descrição das Funcionalidades

1) Quadro Kanban Jurídico (Frontend)

- Visualização de colunas de fluxo:
    - Novos Casos
    - Em Andamento
    - Aguardando
    - Concluídos
- Movimentação de cartões entre colunas por drag-and-drop;
- Criação de novo processo/caso por formulário estruturado;
- Edição de processo/caso existente;
- Exclusão com confirmação de segurança;
- Campos de classificação:
    - Área do direito (Cível, Penal, Trabalhista, Tributário, Família)
    - Urgência/Prioridade (Alta, Média, Prazo Hoje, Liminar)
    - Instância (1º Grau, 2º Grau, STJ/STF)
    - Status do processo

- **Persistência local dos cartões no navegador por meio de localStorage**, permitindo trabalho offline e retenção de dados entre sessões.

  Papel na aplicação: Oferecer acompanhamento visual e operacional do andamento das atividades e dos casos jurídicos, com interface intuitiva e funcionalidades de uso imediato.

2) Backend (API – Em desenvolvimento)

2.1Autenticação e Controle de Acesso (API)

- Cadastro de usuário (`POST /auth/register`);
- Login com geração de token JWT (`POST /auth/login`);
- Consulta do usuário autenticado (`GET /auth/me`);
- Segurança stateless com validação de token Bearer nas rotas protegidas;
- Criptografia de senha no backend (Argon2) e tratamento de erros de autenticação.

  Papel na aplicação: Garantir acesso controlado ao sistema e base de identidade de usuários.

1.2 Gestão de Clientes (API)

- Criação de cliente (`POST /customers`);
- Listagem de clientes (`GET /customers`);
- Busca por nome ou e-mail (`GET /customers?query=...`);
- Atualização de cliente (`PUT /customers/{id}`);
- Remoção de cliente (`DELETE /customers/{id}`);
- Validações de dados e unicidade de e-mail com respostas padronizadas de erro.

  Papel na aplicação: Organizar o cadastro e a manutenção dos dados dos clientes atendidos pela organização.

3) Interface Institucional (Landing Page)

- Página de apresentação do escritório com seções institucionais;
- Layout responsivo e foco em comunicação com o público externo;
- Elementos de contato e posicionamento profissional;
- Design moderno com componentes reutilizáveis em CSS puro.

  Papel na aplicação: Apoiar a presença digital e a apresentação institucional do escritório.

Arquitetura do Sistema

Backend (Arquitetura Hexagonal)

A API segue uma separação inspirada em arquitetura hexagonal:

```
api/
├── bootstrap/          # Inicialização e configuração
├── user/              # Domínio de autenticação
│   ├── adapter/inbound/rest    # Controllers HTTP
│   ├── application/service     # Casos de uso
│   └── domain/                 # Modelo de domínio
└── customer/          # Domínio de clientes
    ├── adapter/inbound/rest    # Controllers HTTP
    ├── adapter/outbound/persistence  # Repositories JPA
    ├── application/service     # Casos de uso
    └── domain/                 # Modelo de domínio
```

Fluxo resumido:

1. Controller recebe a requisição HTTP
2. Mapper converte DTOs para o modelo de domínio
3. Service executa as regras de negócio
4. Adapter de persistência conversa com o banco via Spring Data JPA
5. Resposta é devolvida em DTO próprio da camada REST

Frontend Kanban

```
view/kanban/
├── src/
│   ├── api/              # Serviço de processo (localStorage)
│   ├── app/
│   │   ├── components/   # Componentes reutilizáveis
│   │   ├── pages/        # Páginas (Home, NewProcess, EditProcess)
│   │   ├── types/        # Tipos TypeScript
│   │   └── data/         # Dados estáticos
│   └── styles/           # CSS global e tema
├── vite.config.ts
└── package.json
```

A camada de `api` foi refatorada para usar localStorage, permitindo funcionamento 100% local do aplicativo.

Frontend Institucional

```
view/landing/
├── index.html
```

```
├── css/
│   ├── global.css         # Reset e variáveis
│   ├── header.css
│   ├── hero.css
│   ├── about.css
│   ├── lawyer.css
│   ├── services.css
│   ├── differentials.css
│   ├── cta.css
│   ├── footer.css
│   └── responsive.css
```

Execução do Sistema

Via Docker Compose (Recomendado)

```
docker compose up --build
```

- API disponível em `http://localhost:8080`
- Frontend Kanban em `http://localhost:5173`
- PostgreSQL em `localhost:5432`

Desenvolvimento Local

Frontend Kanban:

```
cd view/kanban
npm install
npm run dev
```

API:

```
cd api
mvn -pl bootstrap -am spring-boot:run
```

Considerações Finais

O trabalho apresenta uma solução web aplicada ao contexto jurídico, integrando:

- **Backend seguro** com autenticação JWT e gestão cadastral
- **Acompanhamento visual** de atividades em Kanban com persistência local
- **Apresentação institucional** profissional via landing page
- **Estrutura modular** adequada para manutenção e evolução

A organização do repositório, aliada ao uso de práticas de desenvolvimento incremental e arquitetura de componentes, sustenta um processo consistente, com foco em utilidade prática, impacto organizacional e continuidade evolutiva do sistema.

O projeto evidencia a possibilidade de transferência tecnológica e aplicação de conhecimentos acadêmicos em contextos reais, demonstrando a importância da extensão universitária na solução de problemas concretos das organizações.